Compte Rendu — Atelier Pratique MLOps

Intégration de Git + DVC + MLflow dans un Workflow Reproductible (Classification Iris & Wine)

Étudiant : Mohamed Amine BELGUEROUANI	**Date :** 02 Novembre 2026
Matière : MLOps & Cycle de Vie ML	**Datasets :** Iris (150 lignes) & Wine (178 lignes)
Stack : Python 3.12, Git 2.52, DVC 3.67, MLflow 3.17	**Backend MLflow :** SQLite (sqlite:///mlflow.db)
Dépôt GitHub : https://github.com/anakin-skyguy/atelier-mlops	**Tests & CI :** Pytest (4/4 passés) + GitHub Actions

1. Architecture Générale et Complémentarité Git + DVC + MLflow

Un système de Machine Learning industrialisable dépend simultanément du **code source**, des **données** et des **hyperparamètres d'entraînement**. Cet atelier assemble trois outils spécialisés afin d'assurer une traçabilité et une reproductibilité de bout en bout :

Fonctionnalité MLOps	Git / GitHub	DVC	MLflow
Versioning du code source	Oui (commits, branches)	Non	Non (trace le SHA)
Versioning des datasets	Pointeurs .dvc	Oui (cache + remote)	Métadonnées
Pipeline ML reproductible	dvc.yaml & dvc.lock	Oui (dvc repro / dag)	Tracking associé
Suivi des hyperparamètres	params.yaml	dvc params diff	Oui (log_params)
Suivi des métriques	reports/metrics*.json	dvc metrics show/diff	Oui (log_metrics)
Suivi des artefacts modèles	Exclus (.gitignore)	Oui (sorties outs:)	Oui (log_artifact)
Comparaison visuelle des runs	Non	Comparaison CLI	Oui (UI & graphiques)

2. TP 1 — Gestion du Code Source avec Git et Publication GitHub

Le dépôt a été structuré autour des branches main (versions stables publiées) et develop (intégration continue des travaux), avec **13 commits atomiques** signés sous l'identité de l'étudiant et jalonnés par trois tags annotés (v1.0, v2.0, v3.0).

- **Configuration .gitignore adaptée à DVC :** Exclusion de __pycache__/, .venv/, mlruns/ et mlflow.db, tout en laissant DVC administrer data/raw/.gitignore et models/.gitignore.
- **Normalisation inter-plateformes (.gitattributes) :** L'ajout de * text=auto eol=lf dès l'initialisation évite toute divergence d'empreinte MD5 des dépendances Python entre Windows (CRLF) et Linux Ubuntu (LF) lors de dvc status.

```
6a24c86 (HEAD -> main, tag: v3.0, develop) Documentation complete, captures MLflow UI haute resolution et README
9208786 Extension CI: Ajout des tests qualite donnees/modeles et workflow GitHub Actions
95f77fa TP3 & TP4: Ajout du modele KNN au pipeline DVC et suivi MLflow
adee5f3 TP2: Restauration de la version initiale du dataset Wine (178 lignes)
621a49e TP2: Modification du dataset Wine (echantillon de 100 lignes)
2b6ae97 TP2: Versioning du second dataset Wine (178 lignes) avec DVC
2691ffb (tag: v2.0) Experience Random Forest avec 200 arbres et max_depth=8
a54cbbb (tag: v1.0) Ajout du pipeline MLOps integre DVC et MLflow (Random Forest 100 arbres)
60bf79c Configuration des remotes DVC (local storage et distant HTTPS)
aad8dde Versioning du dataset Iris avec DVC
0559893 Initialisation de DVC
0a6e156 Ajout du script de preparation du second dataset (Wine)
a635cc1 Initialisation du projet Iris
```

3. TP 2 — Versioning des Données, Remotes DVC et Restauration

Deux jeux de données tabulaires ont été générés de façon déterministe et placés sous suivi DVC :

- **Dataset principal Iris (data/raw/iris.csv) :** 150 lignes, 5 colonnes — MD5 4d301abed5efe50eccda350cde38e0eb (2 778 octets).
- **Second dataset Wine (data/raw/wine.csv) :** 178 lignes, 14 colonnes — MD5 2925c4a8cd201c79d97d3d283429ef23 (10 962 octets).
- **Double Remote DVC (Local + HTTPS universel) :** Outre le remote local storage (../dvc-storage), un remote HTTPS par défaut github pointe vers la branche orpheline dédiée dvc-storage hébergeant uniquement le cache adressé par contenu (files/md5/...). Ainsi, dvc pull fonctionne immédiatement sur toute machine tierce et dans GitHub Actions.
- **Expérience d'altération et restauration (TP 2.7) :** Après réduction de wine.csv à 100 lignes et commit du nouveau pointeur wine.csv.dvc, l'exécution de git checkout HEAD~1 -- data/raw/wine.csv.dvc suivie de dvc checkout a restauré intégralement les **178 lignes** avec une correspondance MD5 bit-à-bit exacte.

4. TP 3 — Suivi et Comparaison des Expérimentations avec MLflow

Les entraînements sont enregistrés dans sqlite:///mlflow.db sous l'expérience TP_Iris_Classification (découpage stratifié identique : test_size=0.2 soit 120 train / 30 test, random_state=42) :

Tag Git	Run MLflow	Algorithme & Hyperparamètres	Accuracy	Precision	Recall	F1-Score
v1.0	RandomForest_n100_d5	Random Forest (n=100, depth=5)	**0.9333**	**0.9333**	**0.9333**	**0.9333**
v2.0	RandomForest_n200_d8	Random Forest (n=200, depth=8)	0.9000	0.9024	0.9000	0.8997
v3.0	KNN_k5	K-Nearest Neighbors (k=5)	**1.0000**	**1.0000**	**1.0000**	**1.0000**

Interprétation : Passer de 100 à 200 arbres (max_depth 5 → 8) dégrade l'accuracy de 0.9333 (28/30) à 0.9000 (27/30) par sur-ajustement sur un petit échantillon d'apprentissage (120 individus). Le classifieur KNN_k5 obtient 1.0000 (30/30) grâce à la régularité géométrique des classes d'Iris, mais sur 30 individus de test (1 erreur = 3,33 %), une validation croisée stratifiée resterait nécessaire pour confirmer cet écart.

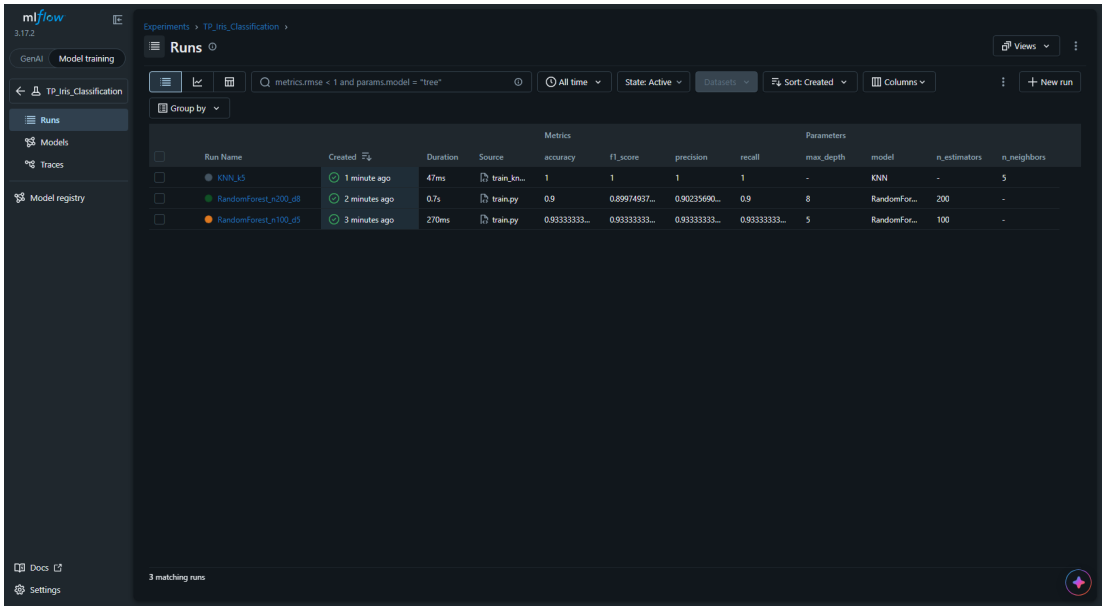


Figure 1 — Interface MLflow UI : tableau des runs de l'expérience TP_Iris_Classification avec métriques et hyperparamètres.

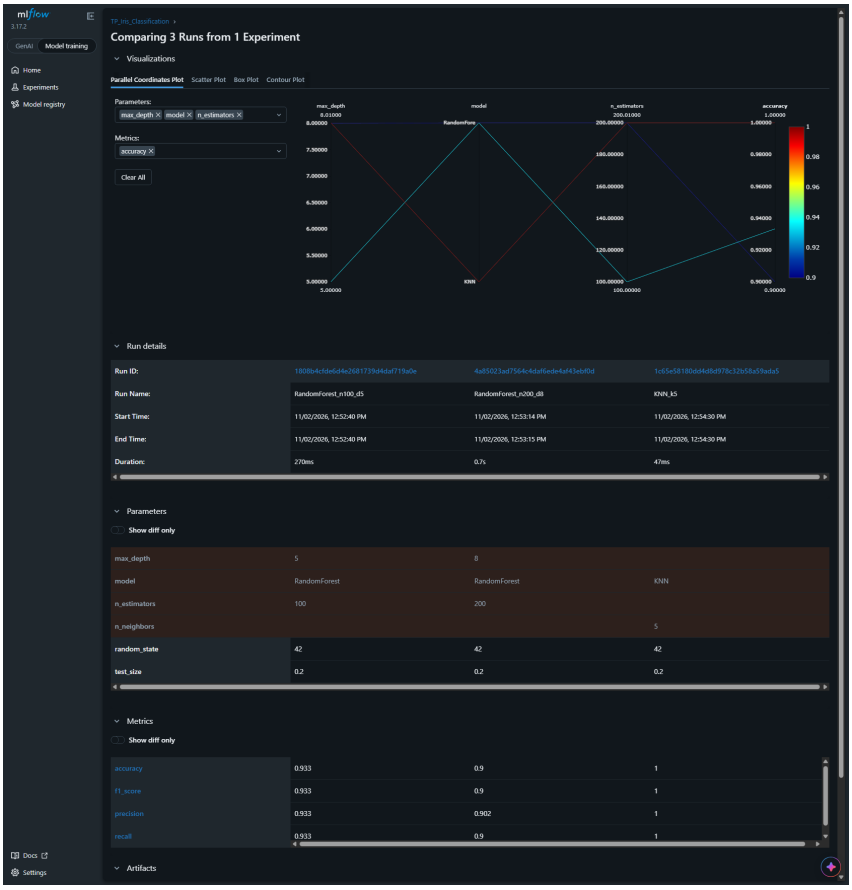


Figure 2 — Module Compare Runs de MLflow : graphique en coordonnées parallèles et tableau comparatif côte à côte des 3 modèles.

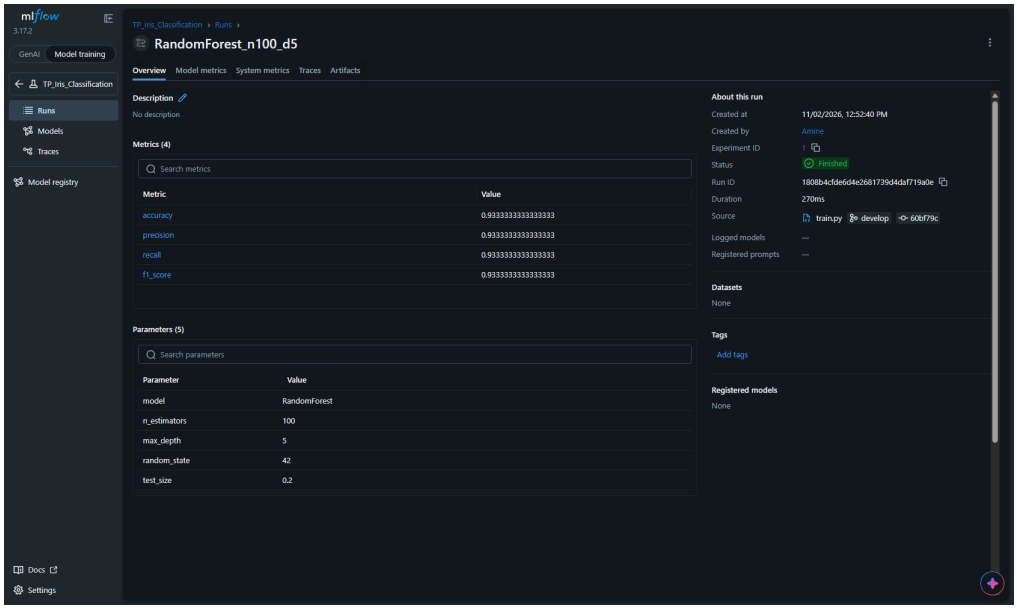


Figure 3 — Fiche détaillée du run RandomForest_n100_d5 (v1.0 : Accuracy = 0.9333, F1 = 0.9333).

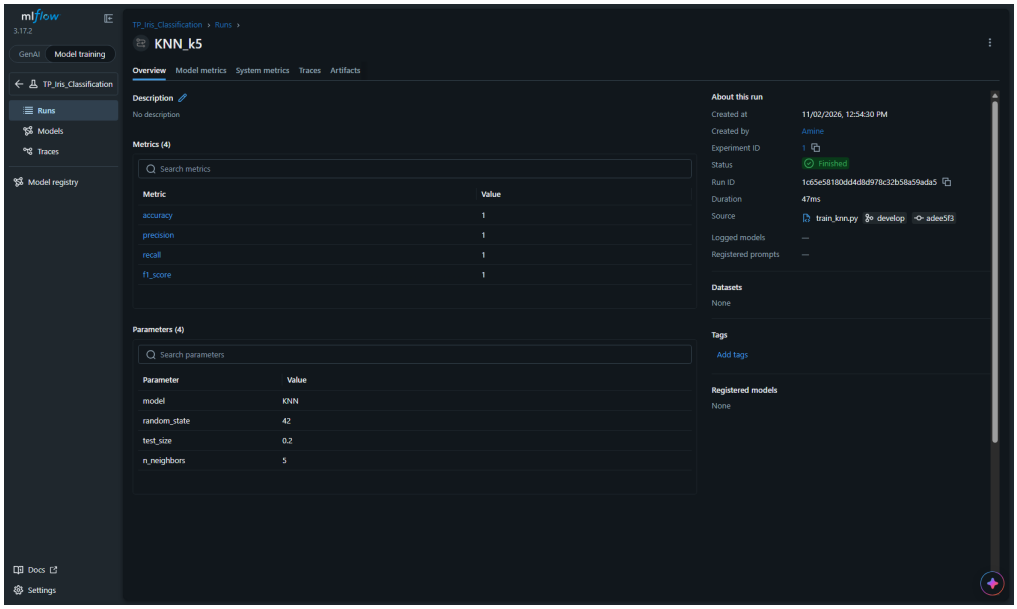


Figure 4 — Fiche détaillée du run KNN_k5 (v3.0 : n_neighbors = 5, Accuracy = 1.0000).

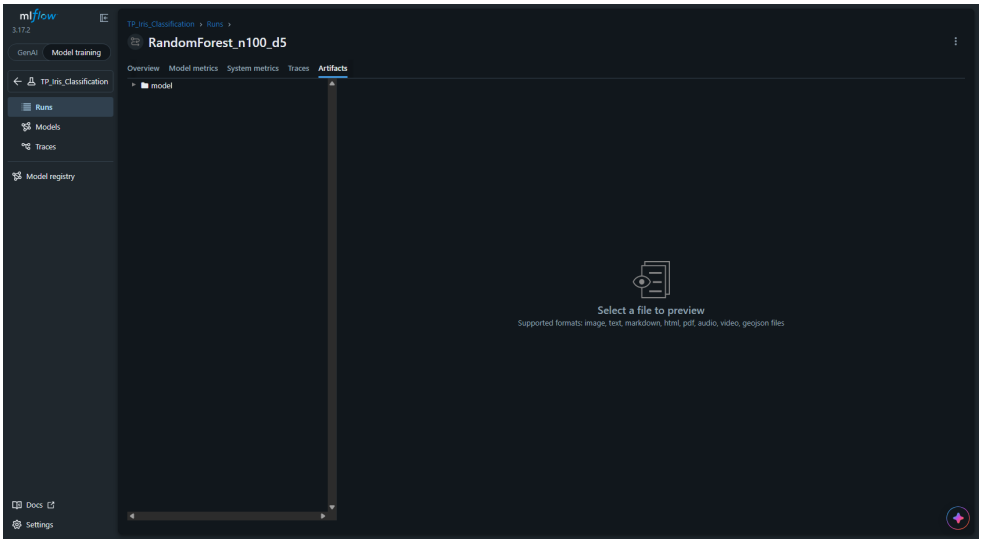


Figure 5 — Onglet Artifacts dans MLflow UI confirmant l'enregistrement du modèle sérialisé (dossier model/).

5. TP 4 — Pipeline Intégré DVC, Reproductibilité et Extension CI/CD

Le fichier déclaratif dvc.yaml orchestre les deux stages d'entraînement (train et train_knn) à partir de data/raw/iris.csv et params.yaml, tandis que dvc.lock fige les empreintes MD5 résolues :

```
$ dvc dag
  +-----------------------+
  | data\raw\iris.csv.dvc |
  +-----------------------+
         **         **
       **             **
      *                 *
+-------+         +-----------+
| train |         | train_knn |
+-------+         +-----------+

$ dvc metrics diff v1.0 v2.0
Path                  Metric     v1.0     v2.0     Change
reports\metrics.json  accuracy   0.93333  0.9      -0.03333
reports\metrics.json  f1_score   0.93333  0.89975  -0.03358
```

5.1. Vérification de restauration et test sur environnement vierge isolé

Un test de bout en bout a été conduit dans un répertoire isolé hors du projet (sans accès à ../dvc-storage) : après git clone https://github.com/anakin-skyguy/atelier-mlops.git, la commande dvc pull télécharge automatiquement les 4 artefacts via HTTPS (iris.csv, wine.csv, model.pkl, knn_model.pkl). Sur main comme après restauration de v1.0 (n=100, d=5), v2.0 (n=200, d=8) et v3.0, dvc status renvoie systématiquement *« Data and pipelines are up to date »*.

5.2. Contrôles qualité automatisés (Pytest) et GitHub Actions

Conformément aux recommandations de la Section 12, le fichier tests/test_mlops.py et le workflow .github/workflows/mlops.yml automatisent dvc pull, dvc repro, dvc metrics show et 4 tests unitaires/qualité (intégrité de iris.csv et wine.csv, cohérence de params.yaml/dvc.lock, inférence des modèles .pkl et respect du seuil d'acceptation accuracy & f1_score >= 0.85 — **4 passed**).

6. Réponses aux Questions de Réflexion (Section 16)

Q15. Quelle est la différence entre Git et DVC ?

Git versionne le code source et les fichiers texte légers de configuration (.py, .yaml, .dvc, dvc.lock). DVC versionne les datasets volumineux et les binaires de modèles en stockant leur contenu dans un cache adressé par hachage MD5 (synchronisé avec un remote local ou cloud) et fournit un moteur de pipeline reproductible.

Q16. Pourquoi éviter de stocker un dataset volumineux directement dans Git ?

Chaque révision d'un fichier massif alourdit définitivement l'historique .git de tous les clones, ralentit les opérations réseau (clone/fetch), dépasse les limites des plateformes (100 Mo/fichier sur GitHub) et produit des diffs textuels inutilisables.

Q17. Quelle est la différence entre dvc add et dvc repro ?

dvc add enregistre un fichier statique existant (comme un dataset brut) dans le cache DVC et génère un pointeur .dvc sans exécuter de code. dvc repro parcourt le graphe de stages défini dans dvc.yaml, compare les empreintes des entrées/paramètres à dvc.lock, ne ré-exécute que les étapes modifiées et met à jour dvc.lock.

Q18. Quel est le rôle du fichier dvc.lock ?

dvc.lock capture l'état exact résolu d'une exécution du pipeline : commandes, valeurs des hyperparamètres, tailles et empreintes MD5 de toutes les dépendances et sorties. Suivi par Git, il lie chaque commit à une version unique et vérifiable des données et des modèles.

Q19. Comment MLflow permet-il de comparer deux expérimentations ?

MLflow enregistre dans son Backend Store (sqlite:///mlflow.db) les paramètres, métriques et artefacts de chaque run. Sa vue Compare aligne côte à côte les hyperparamètres différents, les métriques et trace des graphiques comparatifs (coordonnées parallèles, scatter/box plots), complétée par l'API mlflow.search_runs().

Q20. Comment retrouver les données et le modèle associés à un commit Git précis ?

Il suffit d'exécuter git checkout <commit_ou_tag> (qui restaure dvc.lock et les fichiers .dvc de cet instant), puis dvc checkout (ou dvc pull si les objets sont absents du cache local) pour matérialiser exactement les fichiers de données et le modèle binaire correspondants.

Q21. Pourquoi la reproductibilité est-elle importante dans un projet d'intelligence artificielle ?

Parce qu'un modèle dépend conjointement du code, des données, des hyperparamètres et de l'aléa numérique. La reproductibilité garantit la rigueur expérimentale (isoler l'effet d'un changement), l'auditabilité réglementaire, le rollback rapide en cas de régression en production et la collaboration fiable en équipe.

Q22. Quelle est la différence entre le versioning des modèles avec DVC et leur suivi avec MLflow ?

DVC versionne le fichier physique du modèle comme sortie déterministe du pipeline couplée à un commit Git (orienté reproductibilité et restauration). MLflow journalise l'ensemble des runs exploratoires avec leurs métriques, graphiques et états dans le Model Registry (orienté analyse expérimentale et gouvernance du cycle de vie). Les deux outils sont complémentaires.

7. Récapitulatif des Livrables Attendus (Section 17) et Conclusion

Livrable attendu	Emplacement / Référence vérifiée	Statut
Lien du repository GitHub	https://github.com/anakin-skyguy/atelier-mlops (main, develop, v1.0–v3.0)	**Validé**
Configurations DVC	.dvc/config, data/raw/*.dvc, dvc.yaml, dvc.lock	**Validé**
Code Python & Paramètres	src/prepare_*.py, src/train.py, src/train_knn.py, params.yaml	**Validé**
Captures de l'interface MLflow	6 captures HD dans docs/screenshots/ et intégrées aux Figures 1 à 5	**Validé**
Comparaison d'expérimentations	RF v1.0 (0.9333) vs RF v2.0 (0.9000) vs KNN k=5 (1.0000) — Section 4	**Validé**
Restauration & Reproductibilité	Rollback Wine MD5 + Rollback v1.0/v2.0 + Clone isolé HTTPS + Pytest 4/4	**Validé**

Conclusion : L'intégration conjointe de **Git** (code et pointeurs), **DVC** (données volumineuses, modèles et pipeline déterministe) et **MLflow** (suivi analytique des expérimentations) transforme un simple script d'entraînement Scikit-learn en un véritable pipeline MLOps industrialisable, traçable, auditable et intégralement reproductible sur n'importe quel environnement.